JEE / NEET Examination Platform — Project Overview

> For CTO, CEO, and technical stakeholders. Covers what it is, who uses it, how it works, the technical architecture, current build status, and how to explain each layer.

What Is This?

An **internal online exam platform** built exclusively for our coaching institute. Students take JEE and NEET mock tests on it. Admins manage the exam schedule, student rosters, and results. Think of it as our own private version of NTA's JEE Main portal — tuned exactly to how our institute operates.

It is not a SaaS product. There is no white-labeling, no multi-tenancy, no external customers. It is built, owned, and run by us.

Who Uses It — Three Roles

Role	Who	What They Can Do
Super Admin	Platform owner / head office	Manage all centres, programs, sessions, batches, exam plans across the organization
Admin	Centre-level coordinator	Manage students, upload exams, release results — scoped to their own centre only
Student	Enrolled students	Take exams, view results, view solutions after release

---

The Business Hierarchy (How the Institute is Structured in the System)

```
Program Type  (e.g., JEE 2-Year, NEET 1-Year, Foundation 9/10)
  └── Program  (e.g., JEE 2-Year Batch of 2027)
        ├── Session  (Academic year — 2026-27, 2027-28)
        └── Batch  (Classroom group — "Ranker A", "Achiever B" — tagged to a
Centre)
              └── Enrollment  (One student ↔ one batch)
                    └── Student

Exam Plan  (Full test schedule for a program)
  └── Exam Plan Item  (One scheduled test, e.g., "Full Syllabus Test 3")
        └── Exam Snapshot  (Immutable published paper — once published, never
changed)
              ├── Section  (Physics / Chemistry / Maths, with attempt limits)
              ├── Questions  (With LaTeX support for equations)
              ├── Answer Key  (Hidden until solutions are released)
              └── Attempt  (One per student — scored automatically)
```

```
                    └── Attempt Answer  (Per-question response)
```

Key rule: Adding a new city centre is a data change — you insert one row in the database. No code change needed.

What the Platform Does — Feature List

Exam Management

- Upload exam papers (PDF or manual entry) → system parses questions, options, answer keys
- Publish exams to specific batches (not all students see all exams)
- Define exam windows (students can start any time within a 3-hour window)
- Section-wise question attempt limits (e.g., JEE Main: attempt any 5 out of 10 in Section B)
- Partial marking for MCQ multi-correct (JEE Advanced style)
- Release solutions after scoring — separately from publishing, preserving paper integrity

Student Exam Experience

- **Replicates NTA JEE Main / NEET UI exactly** — same question palette, same section navigation
- 2-step exam start: view instructions → agree → begin (timer starts only after agreement)
- Server-authoritative countdown timer (client display only; server decides if time is up)
- Auto-save every 10 seconds to Redis, flushed to database in background
- Auto-submit if student's browser crashes or session expires
- LaTeX rendering for all equations (KaTeX library)
- Passage-based questions (multiple questions share one reading passage)

Scoring & Analytics

- Automatic scoring as soon as exam is submitted
- Negative marking (JEE penalty) — scores can be negative
- Rank computation: per batch, per centre, per program
- Student performance dashboard with trends across all their exams
- Admin result view with class-level statistics
- Cross-centre analytics for Super Admin

Administration

- Bulk student upload via CSV (creates accounts + enrollments in one shot)

- Student transfer between batches (history preserved)
- Session rollover for new academic year (no re-enrollment needed)
- Audit log — every admin action recorded
- Real-time proctoring event log (tab switches, fullscreen exits)

How a Student Takes an Exam — End to End

```
1. Student logs in → dashboard shows upcoming exams

2. Student clicks "Start Exam"
   → Server checks: is student enrolled? Is exam targeted to their batch?
                    Is it within the exam window?
   → Server returns: instructions page, section summary, duration

3. Student reads instructions → clicks "I Agree & Begin"
   → Server creates the Attempt record and starts the timer
   → Server returns the full exam paper from Redis (fast cache)

4. Student answers questions
   → Every 10 seconds, answers auto-save to Redis
   → Background job flushes Redis state to PostgreSQL every 10 seconds
   → If browser closes, all answers up to last save are preserved

5. Student submits (or time runs out — server auto-submits)
   → All answers written to database in one transaction
   → Scoring job queued via message broker (RabbitMQ)

6. Scoring worker runs
   → Applies marking scheme, partial marking rules, section limits
   → Computes score and ranks
   → Sends notification to student

7. Student views result and (after admin releases solutions) full solutions
```

Technology Stack — What We Built With

Layer	Technology	Why
Web Frontend	Next.js 14 (React)	Admin dashboard + student exam UI, server-rendered, fast
Mobile App	Expo (React Native)	Student app for iOS + Android (in development)
Backend API	Spring Boot (Java)	Proven, type-safe, excellent for high-concurrency exam serving
Database	PostgreSQL	Primary data store — all academic records, attempts, answers
Cache	Redis	Live exam state, autosave buffer, session data

Layer	Technology	Why
Message Queue	RabbitMQ	Async scoring, document parsing, email notifications
Object Storage	S3-compatible	PDF uploads, question images
Connection Pool	HikariCP + PgBouncer	Handle 1500+ concurrent students without exhausting DB connections
Monitoring	Prometheus + Micrometer	Real-time metrics during live exams

---

Code Architecture — How the Codebase Is Organized

```
jee-neet-platform/              ← Single git repository (monorepo)
│
├── backend/                    ← All server-side Java code
│   └── modules/
│       ├── common/             Health checks, error handling, shared utilities
│       ├── auth/               Login, JWT tokens, role-based access
│       ├── academic/           Centres, programs, batches, students,
enrollments
│       ├── exam-authoring/     Creating and publishing exam papers
│       ├── exam-runtime/       Serving live exams to students
│       ├── autosave/           Redis write + database flush pipeline
│       ├── evaluation/         Scoring engine + rank computation
│       ├── analytics/          Precomputed dashboards and reports
│       ├── question-bank/      Question repository with tagging
│       ├── document-parser/    PDF → questions pipeline
│       ├── proctoring/         Tab-switch/fullscreen event logging
│       ├── notifications/      Email/SMS/push notifications
│       └── admin/              Admin panel APIs, audit log
│
├── apps/
│   ├── web/                    ← Next.js web application
│   │   ├── (admin)/            Admin + Super Admin pages
│   │   └── (student)/          Student exam and dashboard pages
│   └── mobile/                 ← Expo React Native app (iOS + Android)
│
└── packages/                   ← Shared code used by both web + mobile
    ├── types/                  TypeScript data shapes matching the API
    ├── api/                    All API call functions (one place, used
everywhere)
    ├── hooks/                  Exam logic: timer, autosave, heartbeat
    └── config/                 Constants, error codes, validation schemas
```

Design principle: Business logic lives in the backend service layer. The frontend is a thin UI — it calls the API and renders results. The shared `packages/` folder means any API change is updated once and both web and mobile get it automatically.

Database Design — 7 Schemas

The PostgreSQL database is split into isolated schemas by domain:

Schema	Contains	Changes How Often
`academic`	Programs, batches, centres, sessions, students, enrollments	Rarely
`runtime`	Published exam snapshots, attempts, answers	Never mutated after publish
`authoring`	Draft exams being created	Frequently during authoring
`auth`	User accounts, roles, tokens	Rarely
`parsing`	Document parsing jobs and output	Per upload
`analytics`	Precomputed report tables	Refreshed by background workers
`audit`	Immutable log of every admin action	Append-only

Why immutable snapshots? Once an exam is published, the `runtime` schema snapshot is frozen. If a question has a typo fixed in authoring, the published exam is unaffected — every student answered the same paper. Results are always explainable.

Web Application — All Pages Built

Super Admin Portal (`/super-admin/...`)

Page	Purpose
Dashboard	Platform-wide stats — all centres, programs, active exams
Centres	Create and manage physical campuses
Programs	Create JEE/NEET programs with session links
Sessions	Manage academic years
Batches	View and manage batches across all centres
Exam Plans	Create and schedule exam papers across programs

Admin Portal (`/admin/...`)

Page	Purpose
Dashboard	Centre-specific stats — students, upcoming exams
Students	List, search, bulk upload via CSV
Student Performance	Individual student's history across all exams
Batches	Manage classroom groups, view batch summary

Page	Purpose
Exam Plans	Upload papers, set sections, publish exams
Exam Upload	PDF upload + question parser interface
Exam Preview	Preview paper before publishing
Exam Results	View class-level results after scoring

Student Portal (`/student/...`)

Page	Purpose
Dashboard	Upcoming exams, recent results
My Exams	List of all available and completed exams
Instructions	Pre-exam instructions page (step 1 of exam start)
Exam Attempt	Full NTA-replica exam UI with palette, timer, sections
Submitted	Confirmation screen after submission
Result	Score, rank, subject breakdown
Solutions	Question-by-question correct answer + solution text
Profile	Student profile and enrolled programs
Performance	Charts showing score trends over time

Security Model

- **JWT-based authentication** — 15-minute access tokens, 7-day refresh tokens
- **Role-based access control** — every API endpoint checks role before processing
- **Centre-scoped Admins** — Admin JWT embeds `centre_id`; all queries automatically filter to that centre. An Admin cannot see another centre's data even if they guess the URL
- **Super Admin** — no scope restriction; sees everything
- **Server-side timer** — students cannot manipulate the countdown. The server always computes `remaining = exam_duration - (now - started_at)`. If a student edits JavaScript in browser DevTools, the server still auto-submits when time is actually up
- **Immutable answer key** — answer keys are stored separately and never sent to the client until the Admin explicitly releases solutions

---

Reliability Design — Why We Won't Lose Exam Data

This was the primary engineering concern. An exam platform that loses answers during submission is unusable.

Risk	How We Handle It
Student browser crashes mid-exam	Auto-save to Redis every 10s; last save is used for scoring
Redis goes down	Circuit breaker (Resilience4j) falls back to direct PostgreSQL write within 5 seconds
RabbitMQ goes down	Scoring job written to `message_outbox` table in the same DB transaction as submission; outbox poller retries until broker recovers
Exam server restarts during live exam	Redis holds live state; student reconnects, gets remaining time from server
Clock skew on client	Irrelevant — server computes all time arithmetic
1500 students submit at once	Async scoring workers; submission is fast (one DB write); scoring happens in background

---

Performance Design — 1500 Concurrent Students

Bottleneck	Solution
Exam paper loading (cold)	Pre-warm Redis cache before exam window opens
Autosave writes	Redis absorbs all writes; PostgreSQL flushed in batch every 10s
DB connections	HikariCP pool + PgBouncer proxy — students never exhaust DB connections
Scoring spike (all submit at once)	RabbitMQ queue smooths the spike; workers scale independently
No joins during exam serving	Exam payload is one Redis key; no database queries while exam is live

---

Build Status — What's Done and What's Next

Backend (Spring Boot) — Phases 1–4 Complete

- ☒ Infrastructure, DB connection, error handling
- ☒ Auth (stub JWT for development, real JWT in production)
- ☒ Academic: centres, programs, sessions, batches, students, enrollments, bulk upload

- ☒ Exam Authoring: exam plans, items, question upload, publish pipeline
- ☒ Exam Runtime: 2-step start, live exam serving, autosave, auto-submit
- ☒ Evaluation: scoring engine, partial marking, negative marking, rank computation
- ☒ Analytics: precomputed reports, performance trends
- ☒ Proctoring events, notifications

Web Frontend (Next.js) — Phases 5.1–5.5 Complete

- ☒ Login and authentication
- ☒ Super Admin portal — all management pages
- ☒ Admin portal — students, batches, exam management, results
- ☒ Student exam UI — NTA JEE Main / NEET replica
- ☒ Student dashboard, performance, results, solutions

Mobile App (Expo) — In Development

- ☐ Student exam experience (iOS + Android)
- ☐ Offline-aware autosave
- ☐ Push notifications

Key Decisions Made (and Why)

1. No SaaS / No multi-tenancy We are not selling this. Authorization scopes by `centre_id` — no per-school logic anywhere. Simpler, faster, less attack surface.

2. Modular monolith (not microservices) Microservices at our scale would add deployment complexity without benefit. The backend is one deployable JAR split into clean modules. We can extract a module to a separate service later if one specific area (e.g., document parsing) needs independent scaling.

3. Immutable exam snapshots Once published, a paper never changes. Attempts reference the published version. Results are always explainable with the exact paper the student saw.

4. Server-side authoritative timer The most important exam integrity decision. The client timer is purely cosmetic (a countdown display). The server computes remaining time from database timestamps. No client-side manipulation can extend an exam.

5. NTA UI replica for students Students preparing for JEE Main and NEET are familiar with the NTA interface. Using the same layout reduces cognitive load during actual exams — they have already practiced on a familiar UI.

6. Transactional outbox for scoring jobs Scoring is written to a `message_outbox` database table in the same transaction as the submission. A background poller publishes to RabbitMQ. This guarantees no exam is submitted without a scoring job being created — even if RabbitMQ is

temporarily unavailable.

Infrastructure — What You Need to Run It

```
PostgreSQL 15    — primary database
Redis 7          — cache and autosave buffer
RabbitMQ 3       — async job queue
S3-compatible    — file storage (exam PDFs, question images)
```

Local development uses Docker Compose (one command starts all four). Production uses managed versions of each (e.g., AWS RDS, ElastiCache, AmazonMQ, S3).

API Surface — 71 Endpoints

The backend exposes a REST API at `/api/v1/`. All responses use a standard envelope:

- Success: `{ status: "success", data: {...}, meta: {...} }`
- Error: `{ status: "error", code: "ERROR_CODE", message: "...", request_id: "..." }`

Module	**Base Path**	**Count**
Auth	`/api/v1/auth/`	4
Academic	`/api/v1/centres`, `/programs`, `/batches`, `/sessions`, `/enrollments`, `/students`	16
Exam Authoring	`/api/v1/exam-plans/`	12
Exam Runtime	`/api/v1/exams/runtime/`	7
Autosave	`/api/v1/exams/autosave/`	3
Evaluation	`/api/v1/scoring/`	3
Analytics	`/api/v1/analytics/`	7
Proctoring	`/api/v1/proctoring/`	4
Notifications	`/api/v1/notifications/`	3
Admin Panel	`/api/v1/admin/`	5
Student Self-Service	`/api/v1/students/me/`	4
Total		**71**

Glossary

Term	Meaning
Snapshot	An immutable published exam paper. Created once, never changed.
Attempt	One student's exam session for one snapshot. One row per student per paper.
Centre	A physical campus location (e.g., Jaipur, Delhi).
Program	A concrete cohort — e.g., "JEE 2-Year, 2027 batch".
Session	An academic year (e.g., 2026–27).
Batch	A classroom group within a program at a centre.
Enrollment	The link between a student and a batch. Can be transferred.
Exam Plan	The full test schedule for a program (e.g., 40 tests over 2 years).
Exam Plan Item	One scheduled test within the plan.
Autosave	Automatic saving of student answers every 10 seconds during the exam.
Outbox	A database table used to reliably queue background jobs (scoring, notifications).
marks_incorrect	Negative marking penalty. Stored as a negative number in the DB; displayed as positive to admins.
Section attempt limit	JEE Main rule: a section has e.g. 10 questions but you can attempt only 5.